Requerimientos No Funcionales

1. Rendimiento

- **Tiempo de respuesta en operaciones CRUD**: Toda operación básica de creación, lectura, actualización o eliminación de registros (ya sea de ranchos, animales, eventos o cualquier entidad principal) debe completarse en un tiempo máximo de 2 segundos cuando se ejecuta en un entorno local estándar (computadora con procesador de gama media, 8 GB de RAM y conexión directa a la base de datos). Este tiempo incluye la ejecución de la consulta, el procesamiento en PHP y la generación de la vista.
- **Consultas avanzadas y búsquedas**: Las operaciones de búsqueda (por ejemplo, buscar un animal por su código de arete, filtrar por rancho, raza o rango de fechas) y los listados paginados deben responder en un máximo de 5 segundos, incluso cuando la tabla de animales contenga hasta 10,000 registros. Se debe garantizar esto mediante el uso obligatorio de índices en columnas frecuentemente consultadas (código de arete, ID de rancho, fecha de registro, etc.).
- **Capacidad de manejo de datos**: La aplicación debe permitir la visualización y manipulación fluida de tablas con hasta 10,000 registros de ganado sin que el usuario perciba lentitud significativa. Esto se logrará implementando paginación obligatoria en todos los listados (máximo 50-100 registros por página) y consultas optimizadas.

2. Escalabilidad

- **Arquitectura modular**: El código debe organizarse siguiendo estrictamente el patrón MVC (Model-View-Controller) o una separación clara de responsabilidades (controladores, modelos, vistas y servicios), de modo que sea posible agregar nuevas funcionalidades (como módulos de vacunación, reproducción, ventas o reportes estadísticos) sin necesidad de modificar grandes porciones del código existente.
- **Capacidad inicial de datos**: La aplicación debe operar sin problemas con una configuración inicial de hasta 5 ranchos diferentes y un máximo de 5,000 cabezas de ganado registradas en total, incluyendo sus historiales asociados (eventos, pesajes, tratamientos, etc.).
- **Preparación para crecimiento futuro**: La base de datos debe diseñarse considerando un posible crecimiento anual del 20-30% en el volumen de registros. Se recomienda el uso de tipos de datos adecuados (por ejemplo VARCHAR normalizado para códigos de aretes) y la creación de índices

compuestos en las columnas más consultadas, permitiendo un escalado vertical sencillo mediante el aumento de recursos del servidor de desarrollo o prueba.

3. Disponibilidad y Confiabilidad

- **Disponibilidad en entorno de desarrollo**: La aplicación debe mantener una disponibilidad mínima del 95% durante las sesiones de prueba y demostración en entorno local, lo que implica que los errores inesperados no deben provocar caídas completas del sistema ni pérdida de funcionalidad prolongada.
- **Manejo de errores**: En caso de fallos (conexión perdida, violación de restricción única, error de validación), la aplicación nunca debe mostrar trazas técnicas, mensajes de PHP o consultas SQL al usuario final. En su lugar, debe presentar mensajes claros, y orientados al usuario (ejemplo: “El código de arete 12345 ya está registrado en otro animal” o “No se pudo guardar el registro. Por favor, inténtelo de nuevo”).
- **Integridad de datos**: Todas las operaciones que modifiquen datos (inserción, actualización o eliminación de animales, eventos o ranchos) deben ejecutarse dentro de transacciones ACID proporcionadas por PostgreSQL. Si cualquier paso de la operación falla, la transacción debe revertirse completamente para evitar estados inconsistentes en la base de datos.

4. Seguridad

- **Autenticación obligatoria**: Todo acceso a funcionalidades de creación, edición o eliminación de datos debe estar protegido por un sistema de login. Las contraseñas deben almacenarse únicamente mediante funciones de hashing seguras, y las sesiones deben gestionarse con opciones seguras.
- **Prevención de inyecciones SQL**: Todas las consultas a la base de datos que involucren datos introducidos por el usuario (especialmente códigos de aretes, nombres, fechas o filtros de búsqueda) deben utilizar exclusivamente prepared statements o consultas parametrizadas mediante PDO o pg_prepare/pg_execute.
- **Control de acceso basado en roles**: Se deben implementar al menos dos roles: “Administrador” (con acceso total a todos los ranchos y funcionalidades) y “Usuario estándar” (con acceso limitado a visualizar y editar únicamente los registros del rancho asignado o propios). Las verificaciones de permisos deben realizarse en cada acción sensible.

- **Auditoría de acciones**: La aplicación debe registrar en una tabla de logs o archivo todas las operaciones sensibles realizadas por los usuarios, incluyendo: ID del usuario, tipo de acción (crear/modificar/eliminar), entidad afectada, fecha/hora exacta y descripción breve. Esto facilita la trazabilidad en caso de errores o demostraciones académicas.

5. Usabilidad

- **Diseño responsive**: La interfaz debe adaptarse correctamente a diferentes tamaños de pantalla, desde dispositivos móviles (mínimo 320px de ancho) hasta monitores de escritorio, utilizando frameworks como Bootstrap. Esto es especialmente importante considerando que los usuarios finales podrían consultar datos desde el campo usando teléfonos o tablets.
- **Experiencia en español y claridad**: Toda la interfaz, mensajes, etiquetas de formularios, errores y ayudas deben estar completamente en español. Los formularios deben incluir tooltips o textos de ayuda contextuales en campos críticos (ejemplo: “Ingrese el código completo del arete, sin espacios”) y validaciones inmediatas en el lado del cliente para evitar envíos innecesarios al servidor.
- **Optimización para conectividad limitada**: Considerando el posible uso en zonas rurales, las páginas deben cargarse rápidamente incluso en conexiones lentas. El tamaño total de cada página (HTML + CSS + PHP + imágenes) debe mantenerse preferiblemente por debajo de 100 KB cuando sea factible, evitando cargas de recursos pesados innecesarios y utilizando compresión de archivos.

6. Mantenibilidad

- **Estándares de código**: Todo el código PHP debe seguir las recomendaciones PSR-1, PSR-12 (estilo) y PSR-4 (autoloading). Se debe mantener una estructura de carpetas clara y consistente (ejemplo: /controllers, /models, /views, /public, /config).
- **Documentación interna y externa**: Cada función o clase importante debe incluir comentarios PHPDoc que describan su propósito, parámetros y valores de retorno. Además, el proyecto debe contar con documentación externa mínima pero completa: un archivo README.md con instrucciones detalladas de instalación, configuración de la base de datos, ejecución del proyecto y un diagrama o descripción textual del esquema de entidades.

- **Control de versiones**: El desarrollo debe realizarse obligatoriamente bajo Git, con commits frecuentes y mensajes descriptivos, y utilizando ramas separadas para nuevas características o correcciones.
- **Pruebas automatizadas**: Se deben implementar pruebas unitarias básicas (usando PHPUnit o similar) que cubran al menos el 70% de las operaciones CRUD principales, las validaciones de entrada y las reglas de negocio críticas (ejemplo: unicidad del código de arete).

7. Portabilidad y Compatibilidad

- **Compatibilidad técnica**: La aplicación debe funcionar correctamente con PHP en versiones 8.0 o superiores y PostgreSQL en versiones 13 o superiores, sin depender de extensiones raras o no estándar.
- **Soporte en navegadores**: La interfaz debe ser completamente funcional y visualmente correcta en las versiones actuales de Google Chrome, Mozilla Firefox y Microsoft Edge. Se recomienda probar también en Safari si hay usuarios con macOS.
- **Independencia de sistema operativo**: El proyecto debe poder ejecutarse sin modificaciones importantes en los tres principales sistemas operativos usados en desarrollo académico: Windows (mediante XAMPP o WAMP), Linux (configuración nativa o Docker) y macOS (mediante Homebrew o configuraciones similares).